

Robustness–Efficiency Trade-offs of INT8 Post-Training Quantization and Quantization-Aware Training on Resource-Constrained Edge Devices

Md. Nurul Alam Siddiquei Ador
Department of Computer Science and Engineering
United International University
Dhaka, Bangladesh
mador223170@bscse.uiu.ac.bd

Abstract—Deploying deep neural networks on Internet of Things (IoT) and edge devices is challenging because of limited memory, computational capability, and energy budgets. Integer quantization is widely used to reduce these costs, with post-training quantization (PTQ) offering a simple conversion pipeline and quantization-aware training (QAT) typically providing stronger accuracy retention at the expense of additional training. Existing studies have extensively compared floating-point and quantized models in terms of accuracy, model size, and inference latency; however, deployment decisions for real-world IoT systems also depend on whether quantized models remain reliable when sensor inputs are degraded by noise, blur, illumination changes, or compression artifacts. This study proposes a robustness-aware evaluation of FP32, INT8 PTQ, and INT8 QAT models using CIFAR-10 for clean evaluation and CIFAR-10-C for common input corruptions. A lightweight custom CNN and MobileNetV2 are considered to represent two levels of model complexity. The models are evaluated on resource-constrained edge hardware using classification accuracy, robustness degradation, model size, peak memory usage, inference latency, throughput, power consumption, and energy per inference. In addition, a degradation-aware PTQ calibration strategy is investigated to determine whether a carefully constructed calibration set can narrow the robustness gap between conventional PTQ and QAT without the retraining cost of QAT. The study is designed to provide deployment-oriented guidance on when PTQ is sufficient, when QAT is justified, and how calibration data can influence robustness in quantized IoT inference.

Index Terms—TinyML, Edge AI, Internet of Things, quantization, post-training quantization, quantization-aware training, INT8, robustness, CIFAR-10, CIFAR-10-C

I. INTRODUCTION

Deep neural networks have achieved strong performance in image classification, object detection, speech processing, and sensor analytics, but their computational and memory requirements often make direct deployment difficult on resource-constrained IoT devices. Edge and TinyML systems must operate under strict limitations in RAM, flash storage, compute throughput, latency, and energy consumption. Consequently, model compression and efficient numerical representations have become central to practical on-device artificial intelligence.

A common neural-network representation uses 32-bit floating-point (FP32) weights and activations. Quantization

reduces numerical precision, for example by representing parameters and activations as 8-bit integers (INT8). Integer-only inference can substantially reduce model memory requirements and can improve execution efficiency on processors and accelerators with optimized integer arithmetic. Jacob et al. demonstrated that carefully designed integer quantization and quantization-aware training can enable efficient integer-only inference while maintaining accuracy close to floating-point models [1].

Two widely used approaches are post-training quantization (PTQ) and quantization-aware training (QAT). PTQ converts a trained floating-point model to lower precision after training, typically using a representative calibration dataset to estimate activation ranges. Its principal advantages are simplicity, low optimization cost, and the absence of retraining. QAT, in contrast, simulates quantization effects during training or fine-tuning, allowing model parameters to adapt to quantization noise. Recent edge-oriented evaluations show that QAT can preserve accuracy better than PTQ in many settings, although the relative advantage depends on model architecture, task, and implementation [2], [3].

Most deployment studies emphasize clean-data accuracy, model size, memory usage, and latency. These measurements are essential but do not fully represent real IoT operating conditions. Camera and vision sensors may experience electronic noise, motion blur, focus errors, illumination changes, low contrast, and compression artifacts. Hendrycks and Dietterich introduced CIFAR-10-C as a standardized benchmark for evaluating neural-network robustness under common corruptions [4]. A model that experiences only a minor clean-accuracy loss after quantization may exhibit a larger performance loss under corrupted inputs; therefore, clean accuracy alone may be insufficient for deciding whether PTQ or QAT is more appropriate for an IoT deployment.

This research focuses on the *robustness–efficiency trade-off* of quantization rather than treating quantization only as a model-size reduction technique. The study compares FP32, INT8 PTQ, and INT8 QAT under both clean and corrupted inputs and measures their performance on actual edge hardware. It further investigates whether degradation-aware PTQ

calibration, constructed from a small representative mixture of clean and degraded samples, can improve corrupted-input performance without requiring QAT retraining.

The main contributions of this study are expected to be as follows:

- A controlled comparison of FP32, INT8 PTQ, and INT8 QAT using the same model architectures and evaluation pipeline.
- A robustness-oriented evaluation using clean CIFAR-10 and corrupted CIFAR-10-C inputs across several realistic corruption categories and severity levels.
- A hardware-level comparison using latency, throughput, peak memory, power, and energy per inference in addition to predictive accuracy.
- An investigation of degradation-aware calibration for PTQ as a low-cost alternative for improving robustness without full quantization-aware retraining.
- Practical guidance for selecting PTQ or QAT according to application robustness requirements and IoT resource constraints.

II. LITERATURE REVIEW

A. Quantization for Efficient Edge Inference

Quantization reduces the precision of neural-network weights and activations to decrease memory footprint and computational cost. For edge deployment, INT8 is particularly attractive because many CPUs, digital signal processors, microcontrollers, and neural accelerators provide optimized integer operations. Jacob et al. proposed an integer-arithmetic-only inference framework in which both weights and activations are represented using 8-bit integers, demonstrating an improved latency–accuracy trade-off for mobile inference [1]. Their work established an important foundation for modern integer quantization pipelines and QAT methods.

The importance of quantization becomes more pronounced in TinyML and in-sensor computing, where memory budgets can be limited to tens or hundreds of kilobytes. Aymone and Pau evaluated FP32, INT8 PTQ, and INT8 QAT variants for highly constrained in-sensor machine-learning workloads. Their benchmark showed that INT8 representations generally reduced memory use and inference latency, while QAT retained accuracy better than PTQ in many of the evaluated cases [2]. This indicates that quantization method selection can influence both predictive performance and deployability.

B. PTQ and QAT on Resource-Constrained Systems

PTQ is attractive because it does not require retraining and can therefore be applied to an existing pretrained model with relatively low engineering cost. However, the quantization ranges inferred from calibration data may not optimally represent all activation distributions. QAT addresses this limitation by inserting simulated quantization operations during training or fine-tuning so that the network can adapt to reduced precision.

Upadhyay et al. compared multiple floating-point precisions and INT8 implementations using both PTQ and QAT across

several convolutional neural networks on edge-oriented hardware. Their results showed substantial reductions in model size and improvements in inference speed after quantization, with limited accuracy degradation in many configurations [3]. Such findings motivate the use of INT8 inference but also demonstrate that quantization behavior is architecture- and hardware-dependent.

Recent TinyML deployment studies additionally emphasize hardware-level metrics such as flash footprint, latency, and energy consumption. For example, Bhushan et al. evaluated quantized TinyML models on low-power edge systems and reported practical deployment measurements including memory footprint, inference latency, and energy per inference [5]. These studies support the need to evaluate quantization as a system-level optimization rather than solely as a change in numerical precision.

C. Robustness Under Common Input Corruptions

Model robustness refers to the ability of a trained model to maintain predictive performance when test inputs differ from the clean training distribution. Hendrycks and Dietterich introduced common-corruption benchmarks including CIFAR-10-C, which applies standardized image degradations such as Gaussian noise, shot noise, defocus blur, motion blur, brightness change, contrast reduction, pixelation, and JPEG compression at multiple severity levels [4]. CIFAR-10-C has subsequently become a widely used benchmark for evaluating robustness under realistic distribution shifts.

For IoT vision systems, corruption robustness is practically important because embedded cameras and communication pipelines are exposed to imperfect sensing and transmission conditions. Nevertheless, many quantization studies evaluate clean-data accuracy without systematically analyzing whether quantization changes sensitivity to corruption. This creates an important deployment question: a PTQ model may be attractive because of its low conversion cost, but if its robustness decreases sharply under sensor degradation, QAT or a better calibration strategy may be preferable.

D. Research Gap

Existing work establishes that quantization can substantially improve model size, memory use, and inference efficiency, and that QAT often preserves clean accuracy more effectively than conventional PTQ [2], [3]. Separately, corruption benchmarks such as CIFAR-10-C provide a systematic framework for measuring robustness [4]. However, there remains room for a focused deployment study that jointly evaluates: (i) clean and corrupted-input performance, (ii) FP32, PTQ, and QAT under a controlled architecture and dataset setup, (iii) real edge-device resource and energy measurements, and (iv) the effect of calibration-set composition on PTQ robustness. This work addresses that intersection and treats robustness as a first-class criterion in selecting a quantization strategy for IoT inference.

III. METHODOLOGY

A. Research Questions

The study is organized around the following research questions:

- 1) **RQ1:** How do INT8 PTQ and INT8 QAT affect clean classification accuracy compared with the FP32 baseline?
- 2) **RQ2:** How does quantization affect model robustness under noise, blur, illumination, contrast, and compression corruptions?
- 3) **RQ3:** What trade-offs do FP32, INT8 PTQ, and INT8 QAT provide in model size, memory usage, latency, throughput, power, and energy per inference on resource-constrained hardware?
- 4) **RQ4:** Can degradation-aware PTQ calibration improve corrupted-input robustness sufficiently to narrow the gap with QAT without requiring retraining?

B. Experimental Design

The experimental design uses two image-classification architectures and four deployment variants. The primary models are: (1) a lightweight custom convolutional neural network (TinyCNN) suitable for constrained devices, and (2) MobileNetV2, representing a widely used efficient architecture for mobile and edge inference. Each architecture is trained as an FP32 baseline using identical train/validation/test splits and fixed random seeds where possible.

The following model variants are evaluated:

- 1) **FP32:** original floating-point baseline.
- 2) **INT8-PTQ:** conventional post-training integer quantization using a clean representative calibration set.
- 3) **INT8-QAT:** quantization-aware fine-tuning followed by INT8 conversion.
- 4) **INT8-RPTQ:** proposed degradation-aware PTQ using a calibration set composed of clean images and a controlled subset of degraded images.

Table I summarizes the planned experiment matrix.

TABLE I
PLANNED EXPERIMENTAL MATRIX

Architecture	FP32	INT8 PTQ	INT8 QAT	INT8-RPTQ
TinyCNN	✓	✓	✓	✓
MobileNetV2	✓	✓	✓	✓

C. Datasets

CIFAR-10 is used for model training and clean-data evaluation. It contains 60,000 RGB images of size 32×32 distributed across ten object classes. The standard split of 50,000 training images and 10,000 test images is used, with a validation subset created from the training partition.

Robustness is evaluated using CIFAR-10-C [4]. Rather than reporting only a single aggregated corruption score, the study analyzes representative corruptions relevant to IoT imaging.

The primary set is planned to include Gaussian noise, shot noise, motion blur, defocus blur, brightness, contrast, and JPEG compression. Multiple severity levels are evaluated in order to measure how rapidly performance degrades as input quality worsens.

No CIFAR-10-C test images used for final robustness evaluation are included directly in model training. For INT8-RPTQ calibration, degradation-aware representative samples are generated from the training/validation domain or otherwise separated from the final test samples to avoid test-set leakage.

D. Model Architectures

1) *TinyCNN*: The custom TinyCNN is designed to provide a compact baseline that can be deployed on constrained hardware. A possible architecture consists of three convolutional blocks with 32, 64, and 128 filters, respectively, followed by batch normalization, ReLU activation, max pooling, global average pooling, and a small fully connected classifier. The exact number of parameters and multiply-accumulate operations (MACs) are reported after implementation.

2) *MobileNetV2*: MobileNetV2 is selected because depth-wise separable convolutions and inverted residual blocks were designed for computationally efficient mobile inference. The classifier head is adapted for the ten CIFAR-10 classes. Input preprocessing and any resizing are kept identical across precision variants to ensure a fair comparison.

E. Training Procedure

All FP32 models are trained using the same optimizer, learning-rate schedule, augmentation policy, batch size, and stopping criterion within each architecture. Recommended starting settings are stochastic gradient descent with momentum or AdamW, cross-entropy loss, and standard CIFAR-10 augmentation consisting of random crop and horizontal flip. Hyperparameters are selected using only training and validation data.

To reduce the influence of random initialization, experiments should be repeated using at least three independent seeds. Mean and standard deviation are reported for software-level accuracy measurements. The best validation checkpoint from each run is used for subsequent quantization.

F. INT8 Post-Training Quantization

Conventional PTQ is applied after FP32 training. A representative calibration subset is sampled from clean training data. Static INT8 quantization parameters are estimated from this set. For a floating-point tensor x , affine quantization can be represented as

$$q = \text{clip} \left(\text{round} \left(\frac{x}{s} \right) + z, q_{\min}, q_{\max} \right), \quad (1)$$

where s is the scale factor, z is the zero-point, and q is the quantized integer value. The corresponding dequantized approximation is

$$\hat{x} = s(q - z). \quad (2)$$

Per-channel weight quantization is preferred where supported, while activations are quantized according to the selected deployment framework and hardware capability.

G. INT8 Quantization-Aware Training

For QAT, fake-quantization operators are inserted during fine-tuning so that forward passes approximate INT8 quantization while trainable parameters remain suitable for gradient-based optimization. The pretrained FP32 checkpoint initializes QAT. A lower learning rate than the original FP32 training is used, and the model is fine-tuned for a fixed number of epochs or until validation performance stabilizes. After QAT, the model is converted to the same INT8 deployment format used for PTQ whenever possible.

H. Proposed Degradation-Aware PTQ Calibration

The proposed INT8-RPTQ variant modifies only the PTQ calibration stage. Instead of using exclusively clean representative images, the calibration set contains a controlled mixture of clean samples and degraded versions derived from the training/validation domain. The purpose is to expose the quantizer to a wider range of activation statistics without modifying model weights.

Let C_{clean} denote clean calibration samples and C_{deg} denote degraded calibration samples. The mixed calibration set is

$$C_{mix} = C_{clean} \cup C_{deg}, \quad (3)$$

where the proportion of degraded samples is treated as an experimental parameter. Candidate ratios such as 0%, 25%, 50%, and 75% degraded samples can be tested in an ablation study. The degradation types used for calibration must be generated independently of the CIFAR-10-C test instances used for final evaluation.

This experiment tests whether robust calibration can improve PTQ performance under corrupted inputs while retaining PTQ's primary advantage: no model retraining.

I. Robustness Evaluation

For each corruption type c and severity level s , classification accuracy $A_{c,s}$ is measured. Clean accuracy is denoted by A_{clean} . Absolute robustness degradation is defined as

$$D_{c,s} = A_{clean} - A_{c,s}. \quad (4)$$

A normalized retention score may also be reported as

$$R_{c,s} = \frac{A_{c,s}}{A_{clean}}. \quad (5)$$

Results are reported per corruption and as macro-averages across the selected corruption set. When appropriate, a mean corruption error or a similar normalized corruption metric can be included for comparability with robustness literature.

J. Hardware Deployment

The quantized models are deployed on at least one resource-constrained edge platform. A practical primary target is a Raspberry Pi-class ARM device for repeatable CPU inference measurements. If available, an MCU-class platform such as ESP32-S3 or an STM32 development board can be included to represent a stricter TinyML environment. Hardware, operating system, runtime, clock configuration, thermal conditions, and software versions are documented.

To ensure fair latency comparisons, inference is performed with a fixed batch size of one, warm-up iterations are excluded from measurement, and a large number of timed runs are collected. CPU frequency scaling and unnecessary background processes should be controlled where practical.

K. Evaluation Metrics

The study reports both machine-learning and system-level metrics:

- Clean classification accuracy and macro F1-score.
- Accuracy/F1 under each corruption and severity level.
- Absolute and relative robustness degradation.
- Model file size and parameter storage.
- Peak RAM usage during inference.
- Mean, median, and tail inference latency (e.g., p50 and p95/p99 where feasible).
- Throughput in inferences per second.
- Average power consumption during inference.
- Energy per inference.
- PTQ conversion/calibration cost and QAT fine-tuning cost.

Energy per inference is calculated as

$$E_{inf} = P_{avg} \times t_{inf}, \quad (6)$$

where P_{avg} is the average inference power and t_{inf} is the measured inference time. Where an external power meter is used, the measurement procedure and sampling rate are documented.

L. Statistical Analysis

For repeated experiments, mean values and standard deviations are reported. Paired comparisons can be used when the same test samples are evaluated by multiple model variants. Confidence intervals or non-parametric tests may be used if metric distributions violate normality assumptions. Effect sizes should accompany significance tests where possible so that practical deployment differences are not reduced to statistical significance alone.

M. Reproducibility

The implementation should report random seeds, framework versions, quantization backend, calibration-set size, QAT schedule, compiler/runtime options, and exact hardware specifications. Source code, trained checkpoints where licensing permits, and scripts used to reproduce latency and robustness evaluation should be released in a public repository if possible.

IV. RESULTS

V. DISCUSSION

VI. THREATS TO VALIDITY

Potential threats include the limited number of architectures, the use of synthetic corruption benchmarks as a proxy for real sensor degradation, device-specific integer acceleration, framework-dependent quantization implementations, and possible differences between laboratory power measurements and battery-powered field deployment. These limitations should be considered when generalizing results to other hardware or application domains.

VII. CONCLUSION

REFERENCES

- [1] B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, H. Adam, and D. Kalenichenko, "Quantization and training of neural networks for efficient integer-arithmetic-only inference," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 2704–2713.
- [2] F. M. Aymone and D. P. Pau, "Benchmarking in-sensor machine learning computing: An extension to the mlcommons-tiny suite," *Information*, vol. 15, no. 11, p. 674, 2024.
- [3] G. Upadhyay, C. S. Prasad, V. Akash, A. K. Singh, R. M. Sai, S. Ghosal, S. Kar, S. H. Shantala, J. T. Lalitkrushna, S. Srividhya, and S. A. Balwantrao, "Enhancing cnn inference time and reducing latency on edge and resource-constrained systems through quantization," in *2024 Second International Conference on Networks, Multimedia and Information Technology (NMITCON)*, 2024.
- [4] D. Hendrycks and T. Dietterich, "Benchmarking neural network robustness to common corruptions and perturbations," *International Conference on Learning Representations (ICLR)*, 2019, cIFAR-10-C and related corruption benchmarks.
- [5] C. M. Bhushan, P. Koppuravuri, N. Prasanthi, F. Gazi, M. M. Hussain, M. Abdussami, A. A. Devi *et al.*, "Deploying tinyml for energy-efficient object detection and communication in low-power edge ai systems," *Scientific Reports*, vol. 15, p. 44299, 2025.